

Fast Global Interconnect Driven 3D Floorplanning

Artur Quiring, Markus Olbrich, and Erich Barke

Institute of Microelectronic Systems

Leibniz Universität Hannover

Appelstraße 4, 30167 Hannover, Germany

Email: {quiring, olbrich, eb}@ims.uni-hannover.de

Abstract—Reduction of interconnect length and thus reduction of power dissipation, and improvement of chip-performance is a key benefit of three-dimensional integrated circuits (3D ICs). However, to profit from 3D, carefully planning global interconnects (e.g. buses or wide IO) already at the floorplanning stage is very important. To this end, we present a new global interconnect driven 3D floorplanner which, different from previous work, simultaneously optimizes global interconnect routes, performs TSV placement, and accounts for fixed-outline floorplanning. For our 3D floorplanner, which is based on Simulated Annealing, we introduce an approach that efficiently guides the optimization process towards a valid and low-cost 3D floorplan solution. Experimental results on GSRC benchmarks demonstrate the efficiency of our tool.

I. INTRODUCTION

In future integrated circuits (ICs), interconnects will more and more dominate overall power consumption and performance. Three-dimensional integrated circuits (3D ICs) offer the opportunity to realize designs with smaller interconnect length compared to 2D ICs and thus allow for more efficient ICs. However, to fully benefit from 3D, carefully planning interconnects is indispensable.

The experiments in [11] reveal that the main advantage of 3D ICs would be the reduction of very long global interconnects, which heavily determine the performance. Furthermore, it would be beneficial to reuse 2D IP modules to reduce design cost for near-term 3D ICs [7]. Both studies indicate that 3D interconnect planning should be already performed at block-level (floorplanning stage), where several 2D modules have to be arranged on various layers, and to be connected by global interconnects (inter-module connections).

Accounting for growing communication demands, interconnect structures like buses, NoCs and wide IO are frequently used to realize global interconnects. They have to be very wide to satisfy communication demands and hence occupy a non-neglectable routing area. Therefore, several research studies [15], [3], [10], [5] have been published, addressing the bus-driven floorplanning problem. All of them, however, consider only 2D designs and thus lack 3D specific design objectives like TSV placement.

Several research studies [6], [7], [9], [14] have been recently published, addressing TSV placement at block-level. Since TSVs occupy non-neglectable device area, they have to be considered as blockage area. In [6] and [7] two methods

have been proposed where TSV placement is performed after floorplanning, using the whitespace of an initial floorplan (post-floorplan TSV placement). Even though the methods are capable of whitespace redistribution and thus allow minor changes to the initial floorplan, they still heavily depend on the quality of the initial floorplan. In [14] and [9] TSV placement is performed directly during floorplanning rather than in a post-floorplan step. The modules can be placed more appropriately so that sufficient whitespace is available to position the TSVs optimally in terms of wirelength and other design objectives. However, the drawback of all mentioned TSV placement aware optimization methods is, that they do not consider routing of global interconnects.

Even though there exist 3D floorplanners [8], [9], [14] which account for interconnect planning, to the best of our knowledge, no 3D floorplanner performs concrete global interconnect routing with TSV placement, and simultaneously considers further important design objectives such as fixed-outline. A simple reuse of previously mentioned approaches is not easily possible since no strategy exists to combine the different approaches with each other.

Our major contributions are as follows:

- We solve the mentioned problem by providing a new global interconnect driven 3D floorplanner, introducing a methodology that allows to simultaneously consider routing of global interconnects, placement of TSVs, and fixed-outline.
- To this end, we add problem specific adaptations to classical Simulated Annealing (SA). We propose for each design objective an analysis function, to guide the SA towards a routable and low-cost 3D floorplan solution.
- We propose a very efficient global routing algorithm.
- We provide our 3D floorplan benchmarks with global interconnect information to allow for comparison with our results.

Experimental results show the efficiency of our 3D floorplanner.

II. PROBLEM FORMULATION

In this paper, the commonly used F2B multi-chip stacking process is assumed [14]. This implicates that a TSV connecting two layers only occupies device area on the upper layer. Furthermore, to simplify routing, it is expected that single global interconnects are hierarchically arranged, e.g., in

buses [15]. They are linked across layers using TSV islands (throughout this paper, we use the term TSV representatively for TSV island). The size of a TSV depends on the width of the respective global interconnect. An illustration of a 3D floorplan with one TSV and two global interconnect routes is shown in Fig. 1.

Based on the preliminary assumptions we define the 3D floorplanning problem as follows. Given:

- A fixed-outline.
- One horizontal and one vertical routing layer for each device layer.
- A set of modules B , where each $b_i \in B$ has a fixed width and a fixed height.
- A set of global interconnects G , where each global interconnect $g_i \in G$ connects a set of modules $g_i = \{b_k, \dots, b_z\}$, has a geometrical width w_{g_i} , and should not exceed a maximal length $l_{i_{max_c}}$.

Find a 3D floorplan, minimizing total global interconnect area, and satisfying the following constraints:

- If a g_i traverses several layers, it is extend by a set of TSVs T_i . A TSV $t_j \in T_i$, connecting two layers, consists of two vertically aligned modules $t_j = \{b_{t_j}, b_{v_j}\}$, where b_{v_j} is the module on the lower layer and b_{t_j} is the module on the upper layer. Only b_{t_j} occupies device area. A global interconnect with TSVs $g_i^* \in G^*$ is defined as $g_i^* = \{g_i, T_i\}$.
- All modules B and all TSVs $T = \bigcup_i T_i$ have to be placed overlap-free within the fixed-outline.
- Two sets, one for horizontal segments S_h and one for vertical segments S_v , have to be determined so that no overlap exists on a device layer between any two segments in S_v and any two segments in S_h . Furthermore, for each $g_i^* \in G^*$ all modules $b_j \in g_i^*$ have to be connected by segments $S_{h_i} \subseteq S_h$ and $S_{v_i} \subseteq S_v$.
- For two modules $b_j, b_k \in g_i^*, b_j \neq b_k$ the accumulated length of all segments connecting b_j and b_k has to be less than the allowed maximal length $l_{i_{max_c}}$.

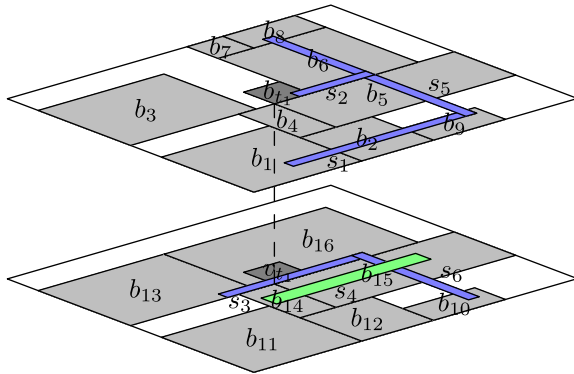


Fig. 1: 3D floorplan with TSVs and global interconnects.

III. GLOBAL INTERCONNECT ROUTING METHOD

Our overall 3D floorplanning optimization algorithm is based on Simulated Annealing (SA). In each SA step, place-

ment of modules and TSVs is altered (using a *move*, *swap*, or *rotate* operation; commonly used in 3D floorplanning) and global interconnect routing is performed. The vertically aligned modules $\{b_{t_i}, b_{v_i}\}$ of a TSV are used to connect the respective global interconnect between two layers. Thus, routing (computation of S_v and S_h) can be performed on each layer separately.

For routing, we use our method *Fast-ST-Routing*. It is based on the approach proposed by Kim and Lim [5], however, we made considerable modifications to significantly improve runtime and solution quality. In Section V experimental results show that *Fast-ST-Routing* performs significantly better than the approach in [5]. The major modifications of *Fast-ST-Routing* are as follows:

- Instead of a congestion based topology generation, we use Steiner trees which can be determined very fast using *FLUTE* [4].
- Before solving the time-consuming Linear Program (LP) in each SA step, our method *Fast-Check* examines in advance if a solution to the LP exists. *Fast-Check* notably improves runtime of the overall optimization algorithm.
- In [5] the routability—which is considered as a term in the cost function—is represented by 1 or 0 (routable or not routable). This oversimplification leads to unsatisfying results when used in an SA-based optimization flow. Therefore, we use a more appropriate routing overflow estimation, computed with our *Fast-Check* method. It allows to better approximate the routability of a floorplan.

Fast-ST-Routing can be divided into four steps. They are described in more detail in the following subsections.

A. Selection of Terminals

To simplify alignment of global interconnects, Hanan grids are used as routing grids [5]. For each global interconnect the respective Hanan grid (only modules of the global interconnect are considered) is determined. An example of a floorplan and a Hanan grid for the global interconnect g_1^* is depicted in Fig. 2a and 2b, respectively. Each grid tile gt_i of the Hanan grid contains an intersection value $gt_{i_{hv}}$, representing the total number of modules which can be connected by a horizontal or a vertical segment with gt_i . To avoid too many bends, it is preferable to connect as many modules and TSVs as possible with one segment. Therefore, the grid tile with the highest $gt_{i_{hv}}$ is selected for each module. For g_1^* in Fig. 2b the selected grid tiles are shaded dark.

Given the selected grid tiles, a topology connecting them has to be found. For this step, the selected tiles are represented by their center points (terminals) k_i as shown in Fig. 2c.

B. Topology Generation

Kim and Lim [5] propose a method for topology generation based on a congestion map to increase the probability of finding a legal routing. This method is very time-consuming, especially when used in the inner loop of an SA. Furthermore, we have observed that routable solutions are not necessarily

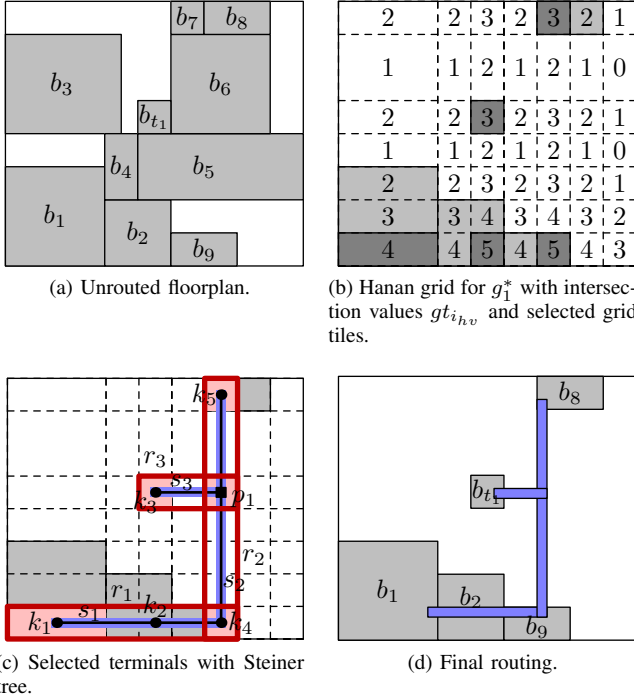


Fig. 2: Global interconnect routing. The modules b_1, b_2, b_8, b_9 and b_{t_1} belong to the global interconnect g_1^* and thus have to be connected. The Hanan grid of g_1^* contains only the modules of g_1^* .

needed in the beginning of the optimization when the floorplan is way off a final solution.

It has been shown that it is sufficient to estimate the interconnect area, in case no legal routing exists, and to consider routability (congestion information) only in the cost function of our SA-based optimization. Therefore, our routing method uses Steiner trees for topology generation, without directly considering congestion information. A Steiner tree can be determined very fast with *FLUTE* [4] and ensures minimal interconnect length for a given set of terminals. Hence, it is a good candidate to reduce interconnect area and maximal interconnect length of a global interconnect. A Steiner tree is illustrated by blue shaded rectangles and an additional Steiner point p_1 in Fig. 2c.

The segments can be obtained from the Steiner tree, where continuous connections like $k_1 - k_2$ and $k_2 - k_4$ are merged together to form one segment s_1 . The starting point and the endpoint of a segment have to be placed within the respective tiles, highlighted as light red shaded rectangles in Fig. 2c. The bounding box of the tiles represents the allowable rectangle r_i (allowable area) for the placement of a segment s_i .

C. Routability Check

Now, the segments determined in the topology generation step have to be placed according to the allowable rectangles. This can be formulated as a Linear Program (LP), with total interconnect area being optimized. To prevent segments of different global interconnects from overlapping, the segments

have to be ordered based on the center coordinates of their allowable rectangles. The ordering is represented in the LP formulation by additional inequalities. For the rectangles r_1 and r_2 of the horizontal segments s_1 and s_2 in Fig. 3, the center y-coordinate of r_1 is smaller than the one of r_2 , hence, s_1 has to be placed below s_2 .

Before solving the LP and thus dissipating runtime, it is preferable to perform a fast routability check in advance. Our method *Fast-Check* is demonstrated for five horizontal segments of different global interconnects in Fig. 3. It can be shortly described as follows:

- 1) Sort all allowable rectangles r_i according to their center x-coordinate (y-coordinate).
- 2) Iterate over all r_i starting with the rectangle which has the lowest x-coordinate (y-coordinate).
 - a) For each r_i , place the respective segment s_i at the lowermost possible position. This is illustrated by the gray shaded rectangles in Fig. 3b.
 - b) If s_i would not fit into r_i , place s_i at the uppermost possible position within r_i , and determine the occurring overlap $\hat{o}_{s_i}$. In Fig. 3b it can be seen that s_6 can not be placed within r_6 , leading to the overlap $\hat{o}_{s_6}$, represented by a dark gray shaded rectangle.
- 3) Return the total overlap $\hat{o} = \sum \hat{o}_{s_i}$ after all r_i have been processed.

If $\hat{o}$ is larger than zero, it is very likely that no solution can be found for the LP, whereas if it is zero, a solution for the LP always exists. Thus, in case of a non-zero $\hat{o}$, the placement of the segments (determined in *Fast-Check*) is used to estimate global interconnect area; no LP is solved. The estimated global interconnect area and $\hat{o}$ are used in the cost function to evaluate the current 3D floorplan solution. If, instead, $\hat{o}$ is zero, an LP formulation is set up and solved like described in the next subsection.

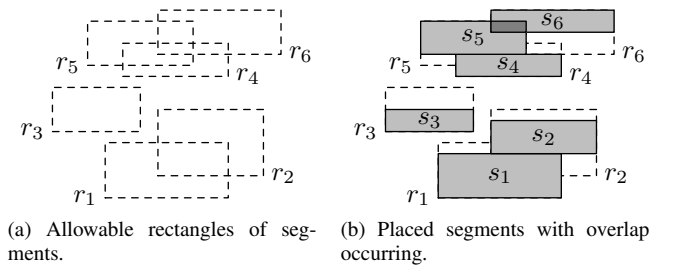


Fig. 3: Routability check.

D. Placement of Routing Segments

To determine the exact segment positions an LP formulation similar to [5] is set up. The formulation consists of the following three constraint groups.

- 1) For each segment s_i , variables x_i^l, y_i^l, x_i^h and y_i^h are created, representing the lower left and upper right coordinates. The ranges for the variables are deduced from the respective tiles (light red shaded rectangles in Fig. 2c).

- 2) Constraints are added, ensuring that two connected segments belonging to the same global interconnect overlap with each other.
- 3) Constraints are added, preventing overlaps between segments belonging to different global interconnects. The constraints are based on the order of the segments (see Section III-C).

Finally, the LP is solved with the state-of-the-art LP solver *SCIP* [1]. The obtained legal routing is used to evaluate the current 3D floorplan solution.

IV. IMPROVEMENT OF OPTIMIZATION ALGORITHM (GUIDED SA)

In [2] it has been shown for the fixed-outline floorplanning problem that it is beneficial to guide the SA, because it is hard for the classical SA approach to find valid solutions. Our 3D floorplanning problem is even harder, since it has to consider the constraints for routability, max length and fixed-outline simultaneously.

Therefore, we have modified the SA perturbation operations to improve the local search characteristics of our SA-based optimization algorithm. Our algorithm uses analysis values, obtained for the design objectives fixed-outline, routability, interconnect area and max length, to favor the selection of certain modules for the next SA perturbation operation, which are good candidates to improve the current 3D floorplan solution. The computation methods for the analysis values are described in the subsequent subsections. Since we use the well-known *B*-Tree* [16] data structure for our 3D floorplanner, all computation methods have been adapted to the *B*-Tree*.

A. Fixed-outline

For fixed-outline floorplanning, the approach described in [13] is used. It computes longest path values lp_i for each module and for each TSV, using a depth-first search in the *B*-Tree*. Modules and TSVs with a large lp_i are suitable to reduce fixed-outline overflow and, therefore, should be selected for the next SA perturbation operation with a higher probability than modules with a low lp_i . The longest path values lp_i are used as analysis values for fixed-outline (FO A.).

B. Routability

To guide SA towards a routable solution, the routing overflow values $\hat{o}_{s_i}$ determined in *Fast-Check* (see Section III-C) are used. Each $\hat{o}_{s_i}$ corresponds to a segment s_i , which in turn corresponds to multiple modules b_j or TSVs $\{b_{t_j}, b_{v_j}\}$. Hence, for each b_j and $\{b_{t_j}, b_{v_j}\}$ all respective $\hat{o}_{s_i}$ can be summed up, leading to the overflow value $\hat{o}_{b_j} = \sum_{s_i \in S_{b_j}} \hat{o}_{s_i}$, where S_{b_j} is the set of segments which connect b_j to the respective global interconnects. Moving a b_j which is responsible for a high routing overflow (i.e. $\hat{o}_{b_j}$ is high) can significantly improve routability. Therefore, the overflow values $\hat{o}_{b_j}$ are used as analysis values for routability (Rout A.).

C. Interconnect Area

Fig. 4 depicts a 3D floorplan with two global interconnects g_1^* and g_2^* . Modules not connected with g_1^* and g_2^* are omitted for clarity. To reduce interconnect area, it is useful to place modules and TSVs next to the centroid of the respective global interconnects, especially if they have a high distance to the centroid. Hence, for each module b_j (and TSV) the centroid c_{b_j} is computed based on the centroids of the respective global interconnects g_i^* of b_j . The centroid c_{g_i} for each global interconnect g_i^* is determined based on the center points of the respective modules and TSVs.

In Fig. 4 the centroids c_{g_1} and c_{g_2} are depicted for g_1^* and g_2^* . Thus, since the module b_{10} is connected with g_1^* and g_2^* the centroid $c_{b_{10}}$ is the centroid of c_{g_1} and c_{g_2} . Moving b_{10} next to $c_{b_{10}}$ would reduce the interconnect area of g_1^* and g_2^* .

Given the current position of a module b_j (center point of b_j) and the centroid c_{b_j} , the distance between these two points is defined as d_j . A module is a good candidate to reduce interconnect area, if it is connected with many global interconnects (number of connections: n_{b_j}), and if it has a high d_j . The values $ar_j = d_j \cdot n_{b_j}$ are used as analysis values for interconnect area (Interc. Area A.).

D. Maximal Length

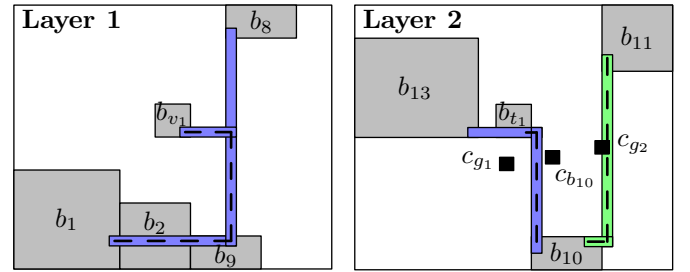


Fig. 4: Routing example for a 3D floorplan with two layers and two global interconnects $g_1^* = \{b_1, b_2, b_8, b_9, b_{10}, b_{13}, b_{t_1}, b_{v_1}\}$ and $g_2^* = \{b_{10}, b_{11}\}$. The centroids for g_1^* and g_2^* are c_{g_1} and c_{g_2} , respectively, and for module b_{10} the centroid is $c_{b_{10}}$.

For a global interconnect g_i^* we define the length $l_{i,j,k}$ between any two modules b_j and b_k based on the segments connecting b_j and b_k ; it is $l_{i,j,k} = \sum len(s_l), s_l \in S_{h_i} \cup S_{v_i}$, where $len(s_l)$ is the length of segment s_l . Furthermore, for a global interconnect g_i^* , the max length $l_{i,max}$, the modules responsible for the max length $L_{i,max}$, and the overflow l_{i_o} are defined as follows:

$$\begin{aligned}
 l_{i,max} &= \max\{l_{i,j,k} \mid b_j, b_k \in g_i^*\} \\
 L_{i,max} &= \{b_j \mid b_j, b_k \in g_i^*, l_{i,j,k} = l_{i,max}\} \\
 l_{i_o} &= \max(0, l_{i,max} - l_{i,max-c}),
 \end{aligned}$$

The overflow o_{b_j} for a module b_j is then defined as follows:

$$o_{b_j} = \sum_{g_i^* \in G^* \mid b_j \in L_{i,max}} l_{i_o}$$

Fig. 4 illustrates a 3D floorplan with two routed global interconnects g_1^* and g_2^* . The dashed line for g_1^* depicts the path between b_1 and b_{10} , representing the maximal length $l_{1_{max}}$, and the dashed line for g_2^* depicts the path between b_{10} and b_{11} , representing $l_{2_{max}}$.

For a legal 3D floorplan the maximal length of each global interconnect g_i^* has to be smaller than $l_{i_{max_c}}$ (i.e., all l_{i_o} have to be zero), where $l_{i_{max_c}}$ is the individual maximal length constraint for g_i^* . In case of a non-zero l_{i_o} it is preferable to move the respective modules b_j with high o_{b_j} close to its centroid c_{b_j} —especially, if the distance d_j between b_j and c_{b_j} is high—in order to reduce the maximal lengths of the respective global interconnects. The values $ml_j = d_j \cdot o_{b_j}$ are used as analysis values for maximal length (Max. Length A.).

E. Combination of Analysis Values

For the combination of the analysis values the approach presented in [13] is used. The analysis values lg_i (FO A.), o_{b_i} (Rout A.), ar_i (Interc. Area A.), and ml_i (Max. Length A.)—having different dimensions—need to be normalized. For instance, each lg_i is divided by the maximum of all lg_i ; that way all lg_i are normalized to the range $[0, 1]$ (normalized values are $\bar{lg}_i$), while the relations are still preserved. The other analysis values are normalized the same way.

Subsequently, the normalized values are summed up $av_i = \bar{lg}_i + \bar{o}_{b_i} + \bar{ar}_i + \bar{ml}_i$, and we obtain a discrete distribution like depicted in Fig. 5. Modules or TSVs with a high av_i have a higher probability to be selected for the next SA perturbation operation than modules with a low av_i .

Using the discrete distribution based on analysis values, our guided SA is more adapted to the 3D floorplanning problem than classical SA which simply uses a uniform distribution. Therefore, it should find a legal 3D floorplan with a good solution quality faster than classical SA.

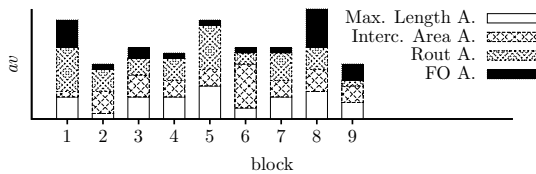


Fig. 5: Discrete distribution used to guide SA.

V. EXPERIMENTAL RESULTS

Our global interconnect driven 3D floorplanner has been implemented in C++. All experiments have been performed on a Linux workstation with an Intel 3.4 GHz CPU and 16 GB of memory. The resulting runtime values are given in seconds. We use the well-known GSRC benchmarks to evaluate our algorithm. However, since they do not contain global interconnect information, we add random global interconnects to the benchmarks (marked with gi). Previous studies have not published their benchmarks, making a comparison difficult. Therefore, we publish all our benchmarks in [12], to allow for comparison with our results.

A. Routing

We have performed experiments to compare the performance of the following routing methods:

- *H-LP* is a reimplement of the floorplanner described in [5]. It uses a congestion-based topology generation.
- *H-LP-Fast-Check* is an extension of *H-LP*. It uses *Fast-Check* rather than only 1 or 0 (routable or not routable) to estimate routability.
- *Fast-ST-Routing* uses Steiner trees for topology generation and *Fast-Check* to estimate routability.
- *Guided Fast-ST-Routing* is an extension of *Fast-ST-Routing*. It uses guided SA based on the analysis values described in Section IV.

Since the approach in [5] only considers 2D floorplanning, we restrict the number of layers to one for these experiments. We set whitespace to 15% and an aspect ratio to one. The cost function consists of interconnect area and routability. The pitch for the congestion map in *H-LP* and *H-LP-Fast-Check* is set to twice the minimal bus width. The weights in the cost function and all other SA specific parameters are set to equal values to allow for a fair comparison.

The routing results are shown in Table I. It can be seen that *H-LP-Fast-Check* produces on average equal results to *H-LP* in terms of interconnect area (Interc. Area is averaged over all legal 3D floorplans) with 10% less runtime, and with a three times higher probability of finding a legal 3D floorplan. The runtime overhead of *H-LP* is basically due to the LP, which has to be solved in each SA step, and the lower probability of finding a legal 3D floorplan is due to the poor consideration of routability in the cost function, misguiding the SA optimization process.

Through the usage of *Fast-ST-Routing* instead of *H-LP-Fast-Check* runtime and interconnect area can be further improved, while probability of finding a legal 3D floorplan is about equal. The runtime overhead of *H-LP-Fast-Check* is basically due to the computation of a congestion map in each SA step. Even though *H-LP* and *H-LP-Fast-Check* have usually a higher probability of finding a routing for a particular 3D floorplan, they perform worse than the Steiner tree based approach, when embedded in an SA based optimization flow.

Comparing *Guided Fast-ST-Routing* with *Fast-ST-Routing*, it can be seen that solution quality can be further improved. *Guided Fast-ST-Routing* finds legal 3D floorplans with a 34% higher probability, and with 9% less interconnect area, with only little runtime overhead.

Overall, the results demonstrate that our routing method *Guided Fast-ST-Routing* is superior to the method *H-LP* presented in [5].

B. Classical SA compared to Guided SA

To the best of our knowledge, there are no other fixed-outline driven 3D floorplanner considering TSV placement and routing. Therefore, we solve the 3D floorplanning problem described in Section II using *Guided Fast-ST-Routing* and *Fast-ST-Routing* to demonstrate the efficiency of our guided

TABLE I: Results for routing-driven floorplanner

Bench- mark	<i>H-LP</i>			<i>H-LP-Fast-Check</i>			<i>Fast-ST-Routing</i>			<i>Guided Fast-ST-Routing</i>		
	Run- time	Interc. Area	Valid (%)	Run- time	Interc. Area	Valid (%)	Run- time	Interc. Area	Valid (%)	Run- time	Interc. Area	Valid (%)
n100-gi	2104	93636	26	1854	92304	72	564	90340	71	611	87103	97
n200-gi	4402	141658	21	3945	132473	60	711	119797	64	759	101537	94
n300-gi	5924	165375	14	5499	162572	63	1050	152504	62	1167	131267	93
Avg. ($\Delta\%$)	0	0	0	-10	-3	2.3 $\times$	-79	-9	240	-78	-18	3.9 $\times$

TABLE II: Results for classical SA and guided SA.

Bench- mark	<i>Fast-ST-Routing</i>			<i>Guided Fast-ST-Routing</i>		
	Run- time	Interc. Area	Valid (%)	Run- time	Interc. Area	Valid (%)
n100-gi	735	79981	65	786	75275	89
n200-gi	813	95438	60	842	89269	87
n300-gi	1245	117385	51	1311	101275	83
Avg. ($\Delta\%$)			0		-9	48

SA compared to classical SA. For each benchmark 100 independent runs have been performed with 15 % whitespace, an aspect ratio of one, and three layers. Fixed-outline, maximal length of global interconnects, and routability have been considered as constraints. Global interconnect area has been considered as optimization goal. The maximal number of SA steps has been the same for *Guided Fast-ST-Routing* and *Fast-ST-Routing*, and it has been set high enough so that both methods show an asymptotic behavior. The results are shown in Tab. II. It can be seen that *Guided Fast-ST-Routing* has a 48 % higher probability of finding a legal 3D floorplan. Furthermore, interconnect area of legal 3D floorplans is on average 9 % lower compared to *Fast-ST-Routing*. Hence, our guided SA approach notably improves solution quality with only little runtime overhead compared to classical SA. An exemplary 3D floorplan with legal routing is shown in Fig. 6.

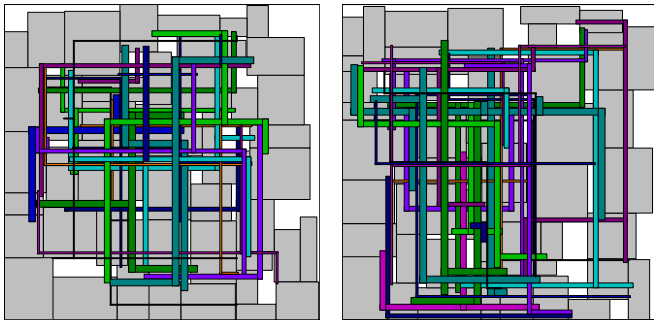


Fig. 6: 3D floorplan (left: layer 1; right: layer 2) with legal routing (n100-gi).

VI. CONCLUSION

In this paper, we have addressed the routing of global interconnects in 3D ICs, which is critical for the IC's power

consumption and performance. We formulate a 3D floorplanning problem, provide appropriate benchmarks, and present a global interconnect driven 3D floorplanner. Experimental results demonstrate that our 3D floorplanner solves the problem efficiently, using a fast Steiner tree based routing method, and a guided SA-based optimization flow.

REFERENCES

- [1] T. Achterberg. SCIP: Solving Constraint Integer Programs. *Mathematical Programming Computation*, 1(1), 2009.
- [2] S. N. Adya and I. L. Markov. Fixed-Outline Floorplanning: Enabling Hierarchical Design. *IEEE Transactions on Very Large Scale Integration Systems*, 11(6), 2003.
- [3] T.-C. Chen and Y.-W. Chang. Modern Floorplanning Based on B*-Tree and Fast Simulated Annealing. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 25(4), 2006.
- [4] C. Chu and Y.-C. Wong. FLUTE: Fast Lookup Table Based Rectilinear Steiner Minimal Tree Algorithm for VLSI Design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 27(1), 2008.
- [5] D. H. Kim and S.-K. Lim. Global Bus Route Optimization with Application to Microarchitectural Design Exploration. *Proceedings of the IEEE International Conference on Computer Design*, 2008.
- [6] D. H. Kim, R. O. Topaloglu, and S.-K. Lim. Block-level 3D IC Design with Through-Silicon-Via Planning. *Proceedings of the Asia and South Pacific Design Automation Conference*, 2012.
- [7] J. Knechtel, I. L. Markov, J. Lienig, and M. Thiele. Multiobjective Optimization of Deadspace, a Critical Resource for 3D-IC Integration. *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2012.
- [8] J. Knechtel, E. Young, and J. Lienig. Planning Massive Interconnects in 3D Chips. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, PP(99), 2015.
- [9] C.-R. Li, W.-K. Mak, and T.-C. Wang. Fast Fixed-Outline 3-D IC Floorplanning With TSV Co-Placement. *IEEE Transactions on Very Large Scale Integration Systems*, 21(3), 2012.
- [10] T. Ma and E. F. Y. Young. TCG-Based Multi-Bend Bus Driven Floorplanning. *Proceedings of the Asia and South Pacific Design Automation Conference*, 2008.
- [11] M. Pathak, Y.-J. Lee, T. Moon, and S.-K. Lim. Through-Silicon-Via Management during 3D Physical Design: When to Add and How Many? *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2010.
- [12] A. Quiring. Global Interconnect Benchmarks. <http://users-ea.ims.uni-hannover.de/quiring/benchmarks>, 2015.
- [13] A. Quiring, M. Olbrich, and E. Barke. Improving 3D-Floorplanning Using Smart Selection Operations in Meta-heuristic Optimization. *Proceedings of the IEEE International 3D Systems Integration Conference*, 2013.
- [14] M.-C. Tsai, T.-C. Wang, and T. Hwang. Through-Silicon Via Planning in 3-D Floorplanning. *IEEE Transactions on Very Large Scale Integration Systems*, 19(8), 2011.
- [15] H. Xiang, X. Tang, and M. D. F. Wong. Bus-Driven Floorplanning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 23(11), 2004.
- [16] Y.-C. C. Yao-Wen, Y.-C. Chang, Y.-W. Chang, G.-M. Wu, and S.-W. Wu. B*-Trees: A New Representation for Non-Slicing Floorplans. *Proceedings of the ACM/IEEE Design Automation Conference*, 2000.